

Practice Set: Destructors in C++

Section 1: Conceptual & Practical Questions (1-10)

Question 1: Define what a **Destructor** is in C++ and state its primary role during an object's lifecycle. Explain the **Resource Leak Problem** that occurs when an object acquiring dynamic heap memory is destroyed without a destructor.

Question 2: Explain the five **Core Characteristics & Constraints** governing destructors in C++ (Name Identity with Tilde, Return Type, Parameter/Overloading Rules, Static Restriction, and Invocation Mechanics).

Question 3: Why is it strictly impossible to **overload** a destructor in C++? How does this contrast with constructors?

Question 4: Compare a **Constructor** versus a **Destructor** across five criteria: Purpose, Invocation Instant, Naming Convention, Parameters/Overloading Support, and Count Allowed Per Class.

Question 5: Detail the **LIFO (Last-In, First-Out) Destruction Rule** governing objects created within the same local block scope on the Call Stack. If three objects are declared sequentially as Object obj1; Object obj2; Object obj3;, trace their precise destruction order.

Question 6: Explain what a **Double Free Error** is in C++. Why does calling a destructor explicitly on a normal stack-allocated object (e.g., `obj.~Complex();`) trigger this catastrophic bug?

Question 7: Explain the core concept of **RAII (Resource Acquisition Is Initialization)** in modern C++. How do constructors and destructors work together to ensure resource safety even in the event of exceptions?

Question 8: Trace the memory state changes of an object containing a pointer (`int *ptr = 0x8000;`) to dynamic heap memory:

When allocated on the stack.

When scope exits without a destructor.

When scope exits with a destructor executing `delete ptr;`.

Question 9: Under what conditions is a destructor invoked **implicitly** by the compiler for stack objects versus heap-allocated objects?

Question 10: Write a complete C++ program that defines a `DynamicArray` class that dynamically allocates heap memory for `$N$` integers in its constructor and safely frees that memory using `delete[]` in its destructor to prevent memory leaks. Demonstrate its automatic cleanup using an enclosed inner block scope in `main()`.

Section 2: Interview & Viva Questions (11-20)

Question 11 (Viva): What symbol must precede the class name to declare a destructor in C++?

Question 12 (Viva): Can a destructor take parameters or return an exit code like `int` or `void`?

Question 13 (Viva): How many destructors can a single C++ class possess?

Question 14 (Viva): Can a destructor be declared as static in C++? Why or why not?

Question 15 (Viva): In what order are local stack objects destroyed when a function block exits?

Question 16 (Viva): What operator must be used inside a destructor to deallocate dynamic memory allocated using `new int[size]`?

Question 17 (Viva): What happens to external resources (such as heap allocations or open file streams) if an object goes out of scope and no destructor is provided to release them?

Question 18 (Viva): Is it safe to invoke an explicit destructor call (e.g., `obj.~ClassName();`) on a stack-allocated object?

Question 19 (Viva): When is the destructor invoked for an object dynamically allocated on the heap via `ClassName *ptr = new ClassName();`?

Question 20 (Viva): What software engineering design idiom couples resource acquisition to constructor execution and resource release to destructor execution?

Answers & Explanations

Answer: 01

Definition: A Destructor is a special instance member function of a class whose primary responsibility is to execute cleanup operations and release acquired resources immediately before an object is destroyed and its memory is reclaimed.

Resource Leak Problem: When an object allocated on the stack exits its scope, its direct member variables are popped from the stack automatically. However, if that object holds a pointer to dynamic heap memory (or open files/sockets), simply reclaiming the stack frame abandons the heap memory without deallocating it, causing a permanent memory leak where the address is lost but the RAM remains occupied.

Answer: 02

Name Matching with Tilde (~): The destructor shares the class name preceded by a tilde symbol (e.g., ~Complex()).

No Return Type: It has no return type (not even void) and cannot return a value.

Takes Zero Arguments: Destructors accept no parameters/arguments, making overloading impossible.

Cannot Be Static: Destructors can never be declared static because they require an individual object instance bound to the this pointer.

Implicit Invocation: They are called automatically by the compiler when an object leaves its scope or when delete is called on a heap pointer.

Answer: 03

Destructors cannot be overloaded because they accept **zero parameters/arguments**.

In C++, function overloading requires variations in parameter count, types, or sequence. Because destructors have no parameter list, there is no signature mechanism for the compiler to distinguish multiple versions. A class can have multiple overloaded constructors, but strictly only **one destructor**.

Answer: 04

Feature	Constructor	Destructor
Purpose	Initializes object state and acquires resources.	Cleans up and releases acquired resources.
Invocation Instant	Invoked immediately when an object is instantiated.	Invoked immediately before an object is destroyed.
Naming Convention	Identical to class name (ClassName()).	Class name preceded by tilde (~ClassName()).
Parameters / Overloading	Accepts arguments; can be overloaded (Default, Parameterized, Copy).	Accepts zero arguments; overloading is strictly impossible.
Count Per Class	Multiple allowed via overloading.	Exactly one per class.

Answer: 05

LIFO Destruction Rule: Local objects allocated on the Call Stack within a single block scope are initialized in sequential declaration order and destroyed in reverse order of creation (Last-In, First-Out).

Trace: For obj1, obj2, obj3 created in order:

Destruction Order: ~obj3 runs first, followed by ~obj2, and finally ~obj1.

Answer: 06

Double Free Error: A runtime corruption bug that occurs when the operating system attempts to deallocate the same heap memory address twice.

Stack Object Hazard: If a developer manually calls `obj.~Complex();`, the destructor executes its cleanup statements. When the enclosing block ends, the compiler still calls the destructor automatically, attempting to delete already freed pointers and causing a program crash.

Answer: 07

RAII (Resource Acquisition Is Initialization): A core C++ idiom that binds resource lifecycle management directly to object lifetime.

Mechanism:

Resources (memory, file streams, mutex locks) are acquired inside the class **constructor** during object creation.

Resources are guaranteed to be released inside the class **destructor** when the object is destroyed.

Because stack-allocated objects are automatically unwound during scope exits (even when exceptions are thrown), resources are safely reclaimed without leaks.

Answer: 08

Stack Allocation: The object `obj` resides on the stack, holding an address pointer `ptr = 0x8000` referencing dynamic heap memory.

Scope Exit without Destructor: The stack memory for `obj` is reclaimed, but heap memory at `0x8000` remains allocated and unreferenced, resulting in a dangling heap memory leak.

Scope Exit with Destructor: The destructor executes `delete ptr;`, deallocating heap address `0x8000` before the stack frame is reclaimed, ensuring zero memory leaks.

Answer: 09

Stack Objects: Invoked implicitly when the object's enclosing block scope exits (function termination or closing brace `}}`).

Heap Objects: Invoked explicitly through the compiler when the developer calls the delete operator on a pointer pointing to that heap object.

Answer: 10

```
#include <iostream>
using namespace std;
class DynamicArray
{private:
    int *data;
    int size;public:
    // Constructor acquires dynamic heap memory
    DynamicArray(int s)
    {
        size = s;
        data = new int[size];
```

```

        cout << "Allocated dynamic heap memory for " << size << " integers." << endl;
    }

    // Destructor frees acquired heap memory
    ~DynamicArray() {
        delete[] data;
        cout << "Destructor freed dynamic heap memory successfully." << endl;
    }
};

int main()
{
    cout << "Creating DynamicArray instance in main..." << endl;
    {
        DynamicArray arr(50); // Memory allocated on heap
        cout << "Working with array inside inner block..." << endl;
    } // Inner scope exits -> Destructor runs automatically -> Heap freed!

    cout << "Continuing main() execution without memory leaks!" << endl;
    return 0;
}

```

Answer: 11

The **tilde (~)** symbol.

Answer: 12

No; destructors take no arguments and have no return type (not even void).

Answer: 13

Exactly one destructor per class.

Answer: 14

No; destructors cannot be static because they must operate on a specific object instance using the implicit this pointer.

Answer: 15

In **reverse order of creation (LIFO / Last-In, First-Out)**.

Answer: 16

The **delete[]** operator (array delete).

Answer: 17

They remain stranded in memory, causing resource and memory leaks because the operating system does not automatically free dynamically acquired heap memory on simple stack frame pops.

Answer: 18

No; calling a destructor explicitly on a stack object causes a Double Free Error when the compiler implicitly invokes it again upon scope exit.

Answer: 19

At the moment the **delete** keyword is applied to that object's pointer.

Answer: 20

RAII (Resource Acquisition Is Initialization)